INGINERIE SOFTWARE 2026–27 · VERSIUNEA DIN 23 OCTOMBRIE 2026

Baremul proiectului 50 de puncte

Proiectul se notează după o singură întrebare: **după ce l-ați schimbat, sistemul a rămas corect, sigur și mentenabil?**

Criteriile urmează cele patru activități ale procesului software din cursul 1: **specification, development, validation, evolution**. Tot proiectul e evoluția sistemului de la MDS; ultima grupă verifică direct schimbarea primită la atelierul 3. Fiecare criteriu trimite la cursul din care vine.

Cum se notează

- **O singură dată**, la prezentarea din sesiune, în intervalul echipei, de **un singur examinator** (Rareș sau Petru). Oralul (30 de puncte, individual) se ține în același interval și se notează separat.
- Se evaluează **ce e în repo la tag-ul** `final`. Termenul pentru tag se anunță odată cu programarea prezentărilor finale. Ce nu e în repo nu există la evaluare.
- Se notează **ce ați făcut după tag-ul** `mds-baseline` și starea finală a sistemului. Ce ați moștenit de la MDS și e bun contează, dacă arătați că l-ați verificat și îl puteți explica.
- **Direcția confirmată la atelierul 1 e reperul de scope.** O puteți ajusta pe parcurs, dacă justificați schimbarea în `DIRECTIE.md`, dar nu o puteți înlocui în întregime. Funcționalitățile care nu țin de direcție sau de o ajustare justificată a ei nu aduc puncte (dar nici nu scad). Nu numărăm linii de cod, commit-uri, PR-uri sau ecrane.
- **Prag:** minimum 25 din 50 la proiect (și minimum 45 de puncte în total, din toate probele).
- **Punctajul echipei** e același pentru toți membrii. Din el se scad, individual, **5 puncte pentru fiecare atelier la care ai lipsit**, cel mult 15. Cum se pot recupera se anunță separat.
- **Notarea nu se face cu AI.** AI-ul poate fi folosit cel mult ca să extragă dovezile din repo; ele se confirmă sau se infirmă la prezentare. Nota o decide examinatorul.

Condiții de evaluare (fără puncte)

1. Tag-urile `mds-baseline` și `final` sunt în repo. Echipele pe un proiect de rezervă au ca baseline tag-ul pus pe codul primit.
2. `DIRECTIE.md` conține direcția confirmată la atelierul 1. Orice schimbare ulterioară se face prin commit în repo, deci are dată și autor, iar motivul e scris.
3. Fiecare membru lucrează din contul propriu de GitHub.
4. Atribuirea AI e în PR-uri sau în mesajele de commit, iar declarația echipei e predată odată cu tag-ul `final`. **Declarația e obligatorie, dar nu se notează.**

> **Recomandare:** faceți din `README.md` indexul tuturor artefactelor proiectului, cu un tabel care spune, pentru fiecare criteriu de mai jos, unde e dovada (fișier, PR, rulare CI). Un criteriu pentru care nu găsim dovada se notează cu 0. De aceea contează tabelul din README.

Pe scurt

Activitatea	Criteriu	Puncte	Cursul (săptămâna)
Specification	1.1 Cerințe verificabile	5	S3, S4
	1.2 Riscuri urmărite	2	S3
	1.3 Modelul de amenințări	3	S11
Development	2.1 Arhitectura, așa cum e	3	S4, S6
	2.2 Componenta LLM (sau integrarea externă) izolată	3	S6, S11
	2.3 Vulnerabilități reparate	4	S11
	2.4 Licențe, date personale, AI	2	S13
	2.5 Build, deploy, observabilitate	3	S12
Validation	3.1 Teste care pot pica	6	S4, S7
	3.2 Accesibilitate	2	S8
	3.3 Porți în CI și fitness function	3	S6, S12
	3.4 Cross-review	3	S7, S9
	3.5 PR-uri revizuite, decizii scrise	3	S6, S7
Evolution	4.1 Change request specificat	4	S4, S14
	4.2 Livrat prin propriile porți	4	S7, S12, S14
	Total	**50**	

Pe cele trei întrebări: **corect** 17 puncte (1.1, 3.1, 3.2, 4.1), **sigur** 13 (1.3, 2.3, 2.4, 4.2), **mentenabil** 15 (2.1, 2.2, 2.5, 3.3, 3.4), iar **procesul** 5 (1.2, 3.5).

1 Specification — 10 puncte

1.1 Cerințe verificabile — CORECT — 5 p

Cursurile S3, S4

Ce verificăm. Cerințele direcției voastre (5–10, din `DIRECTIE.md`, secțiunea 7, ținute la zi). Fiecare are ID, prioritate MoSCoW, tip (funcțională; nefuncțională product, organizational sau external), stakeholderul care a cerut-o și cine a confirmat-o, rationale și felul în care se verifică. Cerințele au trecut prin cele cinci verificări: validity, consistency, completeness, realism, verifiability.

Dovezi. `DIRECTIE.md` (sau `docs/cerinte.md`) și istoricul lui în git.

p	Nivel
0	Lipsesc sau sunt obiective vagi („să fie rapid”).
2	Sunt scrise, dar o parte nu se pot verifica sau nu au un stakeholder.
4	Toate cerințele *must* sunt verificabile, au stakeholder, confirmare și rationale. Conflictele găsite sunt rezolvate.
5	În plus, fiecare schimbare de după atelierul 1 are motivul și pe cine a decis-o. Nicio cerință nu o contrazice pe alta.

1.2 Riscuri urmărite PROCES — 2 p

Cursul S3

Ce verificăm. Riscurile din `DIRECTIE.md` (secțiunea 6) au fost urmărite: ce s-a întâmplat cu fiecare și dacă strategia a funcționat. Riscurile apărute pe drum sunt adăugate.

Dovezi. `DIRECTIE.md`, secțiunea 6, și istoricul ei.

p	Nivel
0	Tabelul lipsește sau a rămas cum era la atelierul 1.
1	E actualizat, dar nu spune ce s-a întâmplat efectiv.
2	Fiecare risc are evoluția și efectul strategiei. Riscurile noi au scor și strategie.

1.3 Modelul de amenințări SIGUR — 3 p

Cursul S11

Ce verificăm. Activele sistemului vostru, amenințările și misuse cases, inclusiv pentru componenta LLM și pentru datele personale. Fiecare amenințare are o evaluare a riscului.

Dovezi. `docs/securitate.md`.

p	Nivel
0	Lipsește sau e generic (s-ar potrivi oricărui sistem).
1	Amenințările sunt reale, dar lipsesc activele sau evaluarea riscului.
2	Complet pentru sistem, dar fără componenta LLM.
3	Complet, inclusiv componenta LLM, cu o prioritizare care explică ce reparați la 2.3.

2 Development — 15 puncte

2.1 Arhitectura, așa cum e MENTENABIL — 3 p

Cursurile S4, S6

Ce verificăm. Descrierea arhitecturii reale: componentele, dependențele dintre ele, componenta LLM și serviciile externe, stilul arhitectural. Primele trei architecture characteristics, fiecare cu cerința sau domain concern-ul din care vine.

Dovezi. `docs/arhitectura.md`, cu diagrama ca text (Mermaid e suficient).

p	Nivel
0	Lipsește sau descrie un sistem ideal, nu pe cel din repo.
1	Diagrama există, dar în puncte importante nu se potrivește cu codul, sau lipsesc caracteristicile.
2	Se potrivește cu codul. Stilul și top 3 sunt numite, cu sursa lor.
3	În plus, relațiile verificate în cod sunt separate de cele deduse, iar documentul e la zi după change request.

2.2 Componenta LLM (sau integrarea externă) izolată MENTENABIL — 3 p

Cursurile S6, S11

Ce verificăm. Puteți păstra, schimba sau scoate componenta LLM de la MDS. Dacă sistemul are o componentă LLM, ea stă în spatele unei interfețe. În CI rulează pe un stub, deci testele nu apelează un API plătit. La timeout sau la eroare, sistemul are un comportament definit (fallback). Intrările venite din afară nu ajung necontrolat în prompt sau în tool-uri. Dacă sistemul nu are (sau nu mai are) componentă LLM, criteriul se aplică celei mai importante integrări externe.

Dovezi. Codul interfeței, stub-ul, testele.

p	Nivel
0	Modelul (sau serviciul extern) e apelat direct, din mai multe locuri, iar testele depind de el.
1	Interfață și stub, dar fără un comportament definit la eroare.
2	Interfață, stub și fallback, toate testate.
3	În plus, un test arată că un răspuns invalid nu strică sistemul; cu LLM, și că o încercare de prompt injection nu îl strică.

2.3 Vulnerabilități reparate SIGUR — 4 p

Cursul S11

Ce verificăm. Cel puțin trei probleme de securitate, din modelul de amenințări sau din scanări, reparate prin PR-uri. Fiecare reparație are un test care ar fi picat înainte de ea. Scanarea secretelor și a dependențelor rulează în CI. Secretele găsite în istoric au fost rotite.

Dovezi. PR-urile, testele, rulările CI, `docs/securitate.md`.

p	Nivel
0	Nicio reparație cu dovadă.
1	Una sau două reparații, sau reparații fără test.
3	Trei reparații cu test, alese după prioritățile din modelul de amenințări.
4	În plus, scanările rulează în CI, iar ce găsesc e rezolvat sau justificat.

2.4 Licențe, date personale, AI SIGUR — 2 p

Cursul S13

Ce verificăm. Licența proiectului e acceptată de coautori, iar licențele dependențelor sunt verificate. E scris ce date personale prelucrează sistemul și unde ajung, inclusiv la furnizorul modelului. Utilizatorul află că interacționează cu o funcție AI.

Dovezi. `LICENSE`, `docs/conformitate.md` (o pagină).

p	Nivel
0	Lipsește.
1	Parțial: doar licențele sau doar datele.
2	Complet și în acord cu ce face codul.

2.5 Build, deploy, observabilitate MENTENABIL — 3 p

Cursul S12

Ce verificăm. Sistemul pornește pe o mașină curată, cu o comandă documentată (de exemplu, un container). Are loguri structurate, un health endpoint și o măsură de disponibilitate sau de latență, cu prag declarat. Fiecare apel la model lasă o urmă: modelul, tokenii, latența.

Dovezi. README, fișierele de build și deploy, un exemplu de log, măsura.

p	Nivel
0	Nu pornește urmând README-ul.
1	Pornește reproductibil, dar nu are observabilitate.
2	În plus, loguri structurate și health endpoint.
3	În plus, măsura cu prag și urma apelurilor la model.

3 Validation — 17 puncte

3.1 Teste care pot pica CORECT — 6 p

Cursurile S4, S7

Ce verificăm. Cerințele *must* au criterii de acceptare care rulează ca teste, cu ID-ul cerinței în numele sau în descrierea testului. Testele rulează în CI. Arătați că testele pot pica: scorul de mutație pe cel puțin un modul important sau mutații introduse intenționat și prinse.

Dovezi. Testele, rularea CI pe `final`, raportul de mutation testing, cifrele înainte și după.

p	Nivel
0	Nu există teste pentru cerințe sau testele nu rulează în CI.
2	Teste pentru o parte din cerințele *must*. Nicio dovadă că pot pica.
4	Toate cerințele *must* au teste legate prin ID, care rulează în CI. Pe un modul există dovada că testele pot pica.
6	În plus, testele slabe găsite (mutanții care au supraviețuit) au fost întărite, cu cifrele înainte și după.

3.2 Accesibilitate CORECT — 2 p

Cursul S8

Ce verificăm. Un flux critic verificat cu o unealtă automată și manual. Problemele găsite sunt reparate sau justificate. Dacă sistemul nu are interfață grafică, stabilim la atelierul 1 ce verificați în loc.

Dovezi. Raportul înainte și după, PR-urile cu reparațiile.

p	Nivel
0	Neverificat.
1	Verificat, problemele sunt listate, dar nu reparate.
2	Reparate sau justificate, apoi reverificate.

3.3 Porți în CI și fitness function MENTENABIL — 3 p

Cursurile S6, S12

Ce verificăm. CI-ul blochează merge-ul când pică o poartă: teste, lint, un prag de acoperire sau de mutație, scanări. Cel puțin o fitness function pentru una dintre primele trei architecture characteristics, automată și cu prag.

Dovezi. `.github/workflows/`, regula de protecție a branch-ului, rulările CI.

p	Nivel
0	CI-ul lipsește sau doar construiește proiectul.
1	Porțile rulează, dar nu blochează nimic.
2	Porțile blochează merge-ul. Fitness function lipsește sau se rulează de mână.
3	În plus, o fitness function automată, cu prag, legată de o caracteristică din top 3.

3.4 Cross-review MENTENABIL — 3 p

Cursurile S7, S9

Ce verificăm. Review-ul dat altei echipe (comentarii concrete, pe cod) și răspunsul vostru la review-ul primit: ce ați schimbat, ce ați respins și de ce.

Dovezi. Comentariile din PR-ul celeilalte echipe, răspunsurile din PR-ul vostru.

p	Nivel
0	Lipsește.
1	Review dat sau primit, dar fără răspunsuri.
2	Amândouă, cu un răspuns la fiecare comentariu important.
3	În plus, schimbările acceptate sunt făcute prin PR-uri legate de comentarii.

3.5 PR-uri revizuite, decizii scrise PROCES — 3 p

Cursurile S6, S7

Ce verificăm. Schimbările de după `mds-baseline` intră prin PR-uri revizuite de un coleg care nu e autorul lor. Deciziile semnificative (cel mult trei) sunt scrise cu context, decizie și consecințe, în descrierea PR-ului sau în `docs/decizii.md`.

Dovezi. PR-urile, `docs/decizii.md` dacă există.

p	Nivel
0	Commit-uri direct pe `main`, fără review.
1	PR-uri, dar review-uri de formă („LGTM”) sau nicio decizie scrisă.
2	Review-uri reale pe majoritatea PR-urilor. Deciziile sunt scrise.
3	Review-uri reale pe toate PR-urile importante. Fiecare decizie are alternativele luate în calcul și consecințele urmărite.

4 Evolution — 8 puncte

4.1 Change request specificat — CORECT — 4 p

Cursurile S4, S14

Ce verificăm. Cererea primită la atelierul 3: criteriile de acceptare scrise ca teste înainte de cod și analiza de impact (ce componente, ce cerințe, ce teste, ce riscuri atinge). Tag-ul `change-request` e pus la finalul atelierului.

Dovezi. `docs/change-request.md`, testele, tag-ul `change-request`.

p	Nivel
0	Lipsește.
2	Specificată, dar criteriile nu sunt teste sau analiza de impact lipsește.
3	Criterii ca teste, scrise înainte de cod (se vede în istoric), și analiza de impact.
4	În plus, la implementare impactul s-a confirmat sau diferențele sunt explicate.

4.2 Livrat prin propriile porți — SIGUR — 4 p

Cursurile S7, S12, S14

Ce verificăm. Implementarea cererii intră printr-un PR revizuit de un coleg, cu CI-ul verde. Contează porțile pe care le aveți la tag-ul `change-request`, nu cât CI aveați la începutul semestrului: testele și verificările care existau atunci rămân verzi, iar niciuna nu a fost slăbită ca să treacă (teste șterse sau dezactivate, praguri coborâte, scanări oprite). Codul moștenit pe care l-ați atins fără să-l înțelegeți complet are teste de caracterizare.

Dovezi. PR-ul, rulările CI, istoricul fișierelor de CI.

p	Nivel
0	Nelivrat sau livrat ocolind CI-ul.
1	Livrat, dar cu porți slăbite sau teste dezactivate.
3	Livrat prin PR revizuit, cu CI verde și fără regresii.
4	În plus, teste de caracterizare pentru codul moștenit pe care l-ați atins.

Ce nu aduce puncte

- Funcționalități care nu țin de direcție sau de o ajustare justificată a ei, oricât de multe.
- Volum: linii de cod, commit-uri, PR-uri, ecrane.
- Documente frumoase care nu corespund codului.
- Teste care trec, dar nu pot pica.
- Un demo cu ceva ce nu e în tag-ul `final`.

Unde stau dovezile (recomandat)

Fișier	Criterii
`README.md`, indexul artefactelor, cu tabelul criteriu → dovadă	toate
`DIRECTIE.md`	1.1, 1.2
`docs/securitate.md`	1.3, 2.3
`docs/arhitectura.md`	2.1
`docs/conformitate.md`	2.4
`docs/decizii.md` (opțional, sau în PR-uri)	3.5
`docs/change-request.md`	4.1
`.github/workflows/` și testele	2.3, 2.5, 3.1, 3.3, 4.2